

Healthcare AI QA Engineering Platform

Step 1.13 Implementation Report

Project:
CareFlow Health

Step:
STEP 1.13 - Playwright + Cucumber BDD Automation Framework Foundation

Objective:
Establish a production-quality UI automation foundation for the existing frontend using Playwright + Cucumber with TypeScript, deterministic selectors, reporting, and traceability.

Architecture:

- BDD feature layer under automation/features
- Step definition layer under automation/src/steps
- Page Object Model under automation/src/pages
- Hook lifecycle and artifact capture under automation/src/hooks
- Runtime config under automation/src/config
- Structured failure metadata under automation/src/utlis

Technologies:

- Playwright
- Cucumber BDD
- TypeScript
- Node.js tooling

Implemented Framework Areas:

- Multi-browser-ready Playwright configuration (Chromium/Firefox/WebKit)
- Cucumber runner and typed world
- Initial smoke feature suite and step definitions
- Core page objects for login, layout, dashboard, patient search, appointments
- Reporting foundation: Cucumber JSON + HTML report
- Failure evidence foundation: screenshot, trace, video path, metadata JSON
- Deterministic role-based test users mapped to synthetic dataset IDs

Scenarios Automated in this step:

1. Successful login
2. Dashboard access after login
3. Patient navigation and search
4. Appointment navigation
5. Role-based restricted navigation visibility
6. Logout flow

Validation Results:

- npm install (automation): PASS
- npm run typecheck (automation): PASS

- npm run smoke:chromium: PASS (5/5 scenarios)
- npm run smoke:firefox: FAIL (missing Playwright Firefox browser binary)
- npm run smoke:webkit: FAIL (missing Playwright WebKit browser binary)
- npm run report:html: PASS
- npm run lint (frontend): PASS
- npm run build (frontend): PASS

Testability and Traceability:

- Stable data-testid-based selector strategy used
- Scenario tags include traceability IDs: @tc-*, @ts-*, @ac-*
- Output artifacts structured for future AI failure-analysis use

AI Boundary and Readiness:

Implemented/Enabled in Step 1.13:

- AI-assisted automation readiness through structured metadata and deterministic artifacts
- Failure-analysis readiness via machine-readable evidence
- AI quality-gate readiness via report and result artifacts

Planned (not implemented runtime):

- RAG, MCP, tool-calling runtime
- LangGraph runtime
- Agentic and multi-agent runtime
- AI observability runtime

Known Issues:

- Firefox and WebKit execution blocked because browser binaries are not installed.
- Cucumber warns that Node v24 has not been explicitly tested.

Interview Summary:

This step demonstrates automation framework architecture, BDD layering, robust evidence capture, traceability mapping, and readiness for future AI-QA capabilities without over-claiming runtime AI.